## Internationalization

*Internationalization* is the process of designing an application so that it can be adapted to various languages and regions without engineering changes. Sometimes the term internationalization is abbreviated as i18n, because there are 18 letters between the first "i" and the last "n."

An internationalized program has the following characteristics:

- With the addition of localized data, the same executable can run worldwide.
- Textual elements, such as status messages and the GUI component labels, are not hardcoded in the program. Instead they are stored outside the source code and retrieved dynamically.
- Support for new languages does not require recompilation.
- Culturally-dependent data, such as dates and currencies, appear in formats that conform to the end user's region and language.
- It can be localized quickly.

## Localization

*Localization* is the process of adapting software for a specific region or language by adding locale-specific components and translating text. The term localization is often abbreviated as l10n, because there are 10 letters between the "l" and the "n."

The primary task of localization is translating the user interface elements and documentation. Localization involves not only changing the language interaction, but also other relevant changes such as display of numbers, dates, currency, and so on. Other types of data, such as sounds and images, may require localization if they are culturally sensitive. The better internationalized an application is, the easier it is to localize it for a particular language and character encoding scheme.

Internationalization may seem a bit daunting at first. Reading the following sections will help ease you into the subject.

### Before Internationalization

Suppose that you've written a program that displays three messages, as follows:

```
public class NotI18N {

    static public void main(String[] args) {

        System.out.println("Hello.");
        System.out.println("How are you?");
        System.out.println("Goodbye.");
    }
}
```

### After Internationalization

The source code for the internationalized program follows. Notice that the text of the messages is not hardcoded.

```
import java.util.*;

public class I18NSample {

    static public void main(String[] args) {

        String language;
        String country;

        if (args.length != 2) {
            language = new String("en");
            country = new String("US");
        } else {
            language = new String(args[0]);
            country = new String(args[1]);
        }

        Locale currentLocale;
        ResourceBundle messages;

        currentLocale = new Locale(language, country);

        messages = ResourceBundle.getBundle("MessagesBundle", currentLocale);
        System.out.println(messages.getString("greetings"));
        System.out.println(messages.getString("inquiry"));
        System.out.println(messages.getString("farewell"));
    }
}
```

To compile and run this program, you need these source files:

- I18NSample.java
- MessagesBundle.properties
- MessagesBundle_de_DE.properties
- MessagesBundle_en_US.properties
- MessagesBundle_fr_FR.properties

**Running the Sample Program**

The internationalized program is flexible; it allows the end user to specify a language and a country on the command line. In the following example the language code is `fr` (French) and the country code is `FR` (France), so the program displays the messages in French:

```
% java I18NSample fr FR
Bonjour.
Comment allez-vous?
Au revoir.
```

In the next example the language code is `en` (English) and the country code is `US` (United States) so the program displays the messages in English:

```
% java I18NSample en US
Hello.
How are you?
Goodbye.
```

**Define the Locale**

The `Locale` object identifies a particular language and country. The following statement defines a `Locale` for which the language is English and the country is the United States:

```
aLocale = new Locale("en","US");
```

The next example creates `Locale` objects for the French language in Canada and in France:

```
caLocale = new Locale("fr","CA");
frLocale = new Locale("fr","FR");
```

The program is flexible. Instead of using hardcoded language and country codes, the program gets them from the command line at run time:

```
String language = new String(args[0]);
String country = new String(args[1]);
currentLocale = new Locale(language, country);
```